

基于openGauss的银行数据业务系统案例进阶

学校：大连理工大学

学院：软件学院

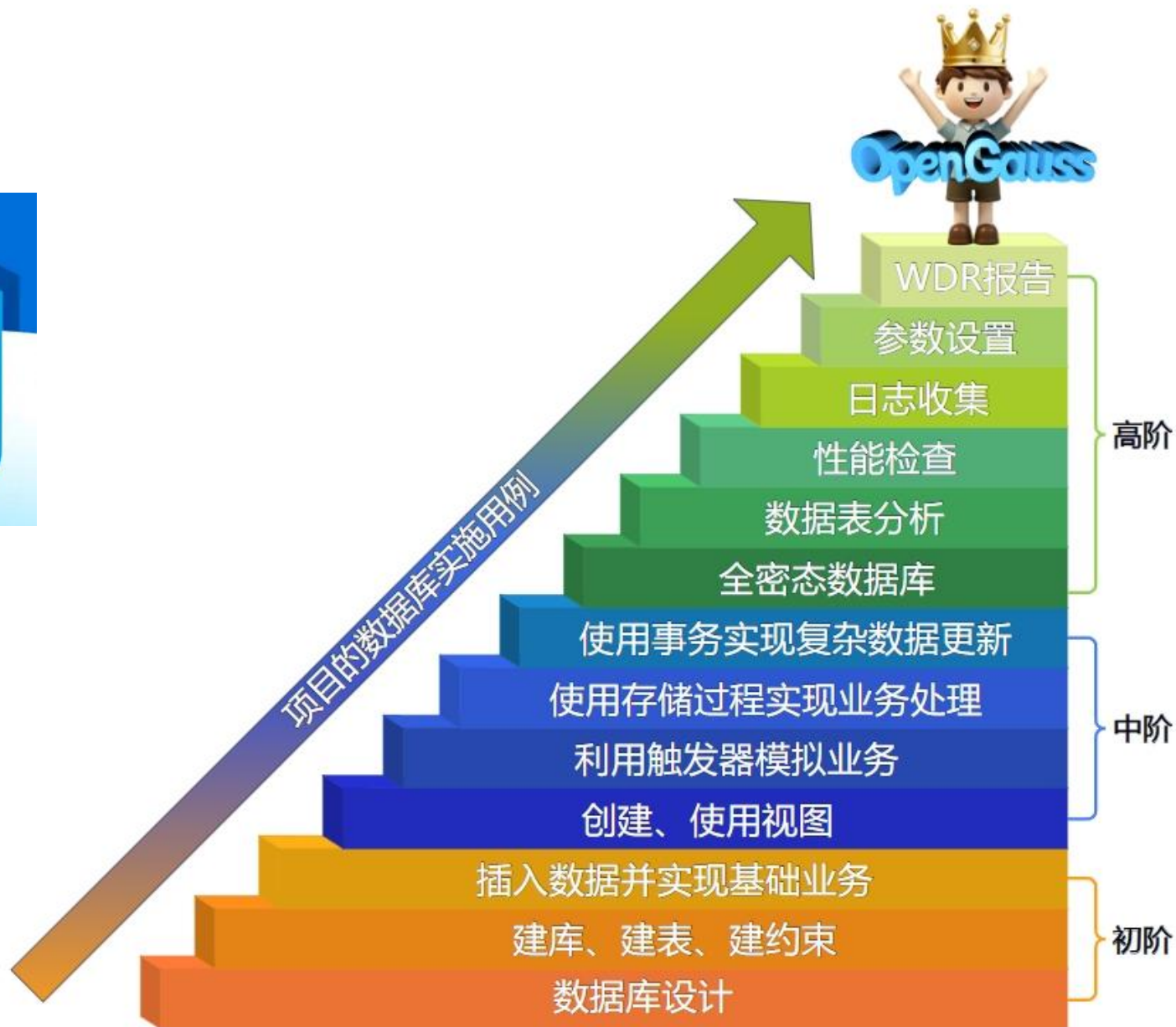
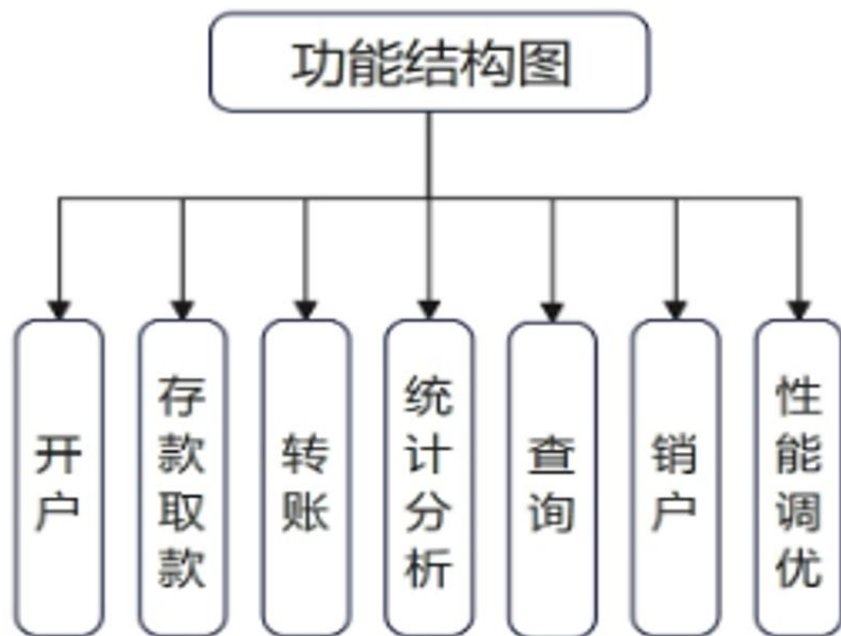
负责人：马瑞新

职务/职称：教师/教授

Security Level:



1 实验案例简介



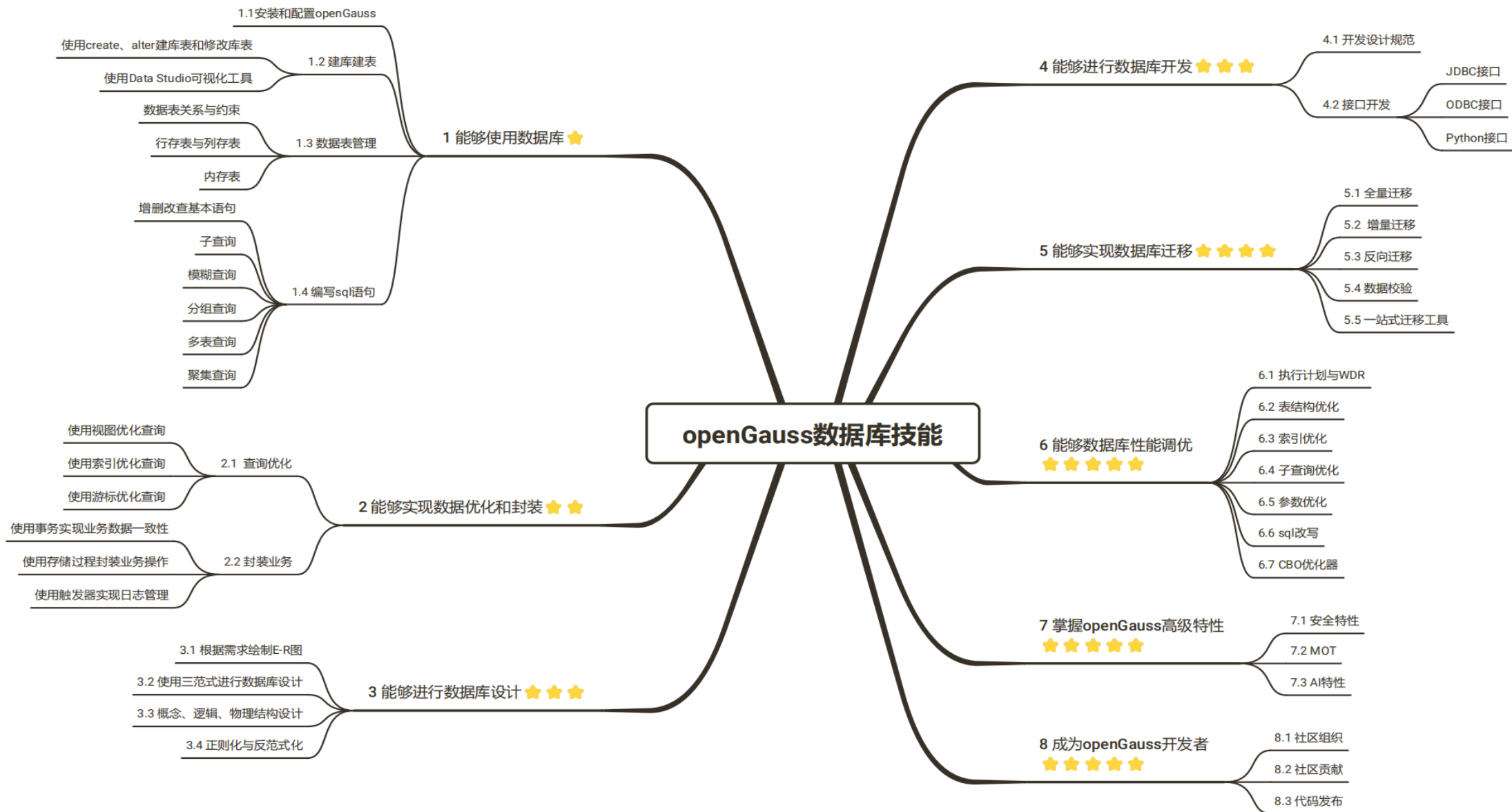
2 知识图谱



知识点内容:

- (1) 学会使用SQL语句创建数据库和表，并添加各种约束。
- (2) 学会进行常见的SQL编程。
- (3) 学会创建事务和事务处理：本银行内账户间的转账。
- (4) 学会使用子查询并做查询优化：查询指定卡号的交易记录。
- (5) 学会创建并使用视图：查询各表时显示友好的中文字段名。
- (6) 学会创建并调用存储过程和游标：包括取款或存款的存储过程、产生随机卡号的存储过程等。
- (7) 学会创建并使用触发器：通过交易记录更新余额。
- (8) 学会对数据库进行性能调优：创建加密表、对数据库进行健康检查等。

3 能力图谱



3 案例资源



 HUAWEI Huawei Talent 



 消息  简体中文 

全部分类  首页 华为ICT学院

我要学习  XZM9B96RR5 



基于银行数据业务系统的openGauss案例进阶

★★★★★ 5分

 65人在学

6小时
学习时长

中文
语言

2025-01-21 08:03
开课时间

2037-12-31 23:59
结课时间

继续学习

课程介绍

课程目录

课程评价

100%

8人评价

好评

中评

差评

我要评价

全部8条评价

内容深入浅出 3

对工作帮助很大 3

内容形式新颖 3

内容质量上乘 3

结构逻辑清晰 3

授课有趣生动 3

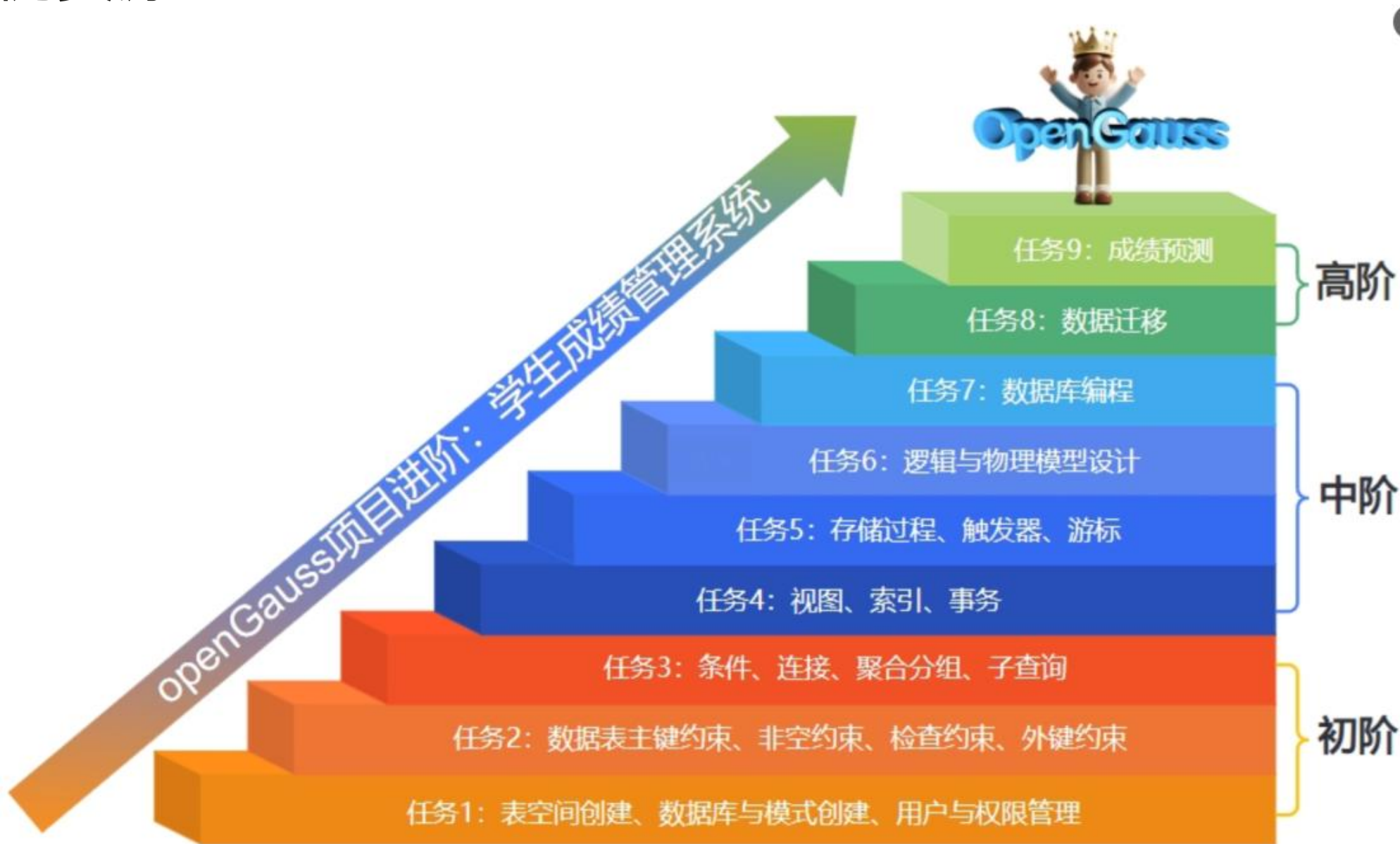
 XZDWHC6MDW

★★★★★

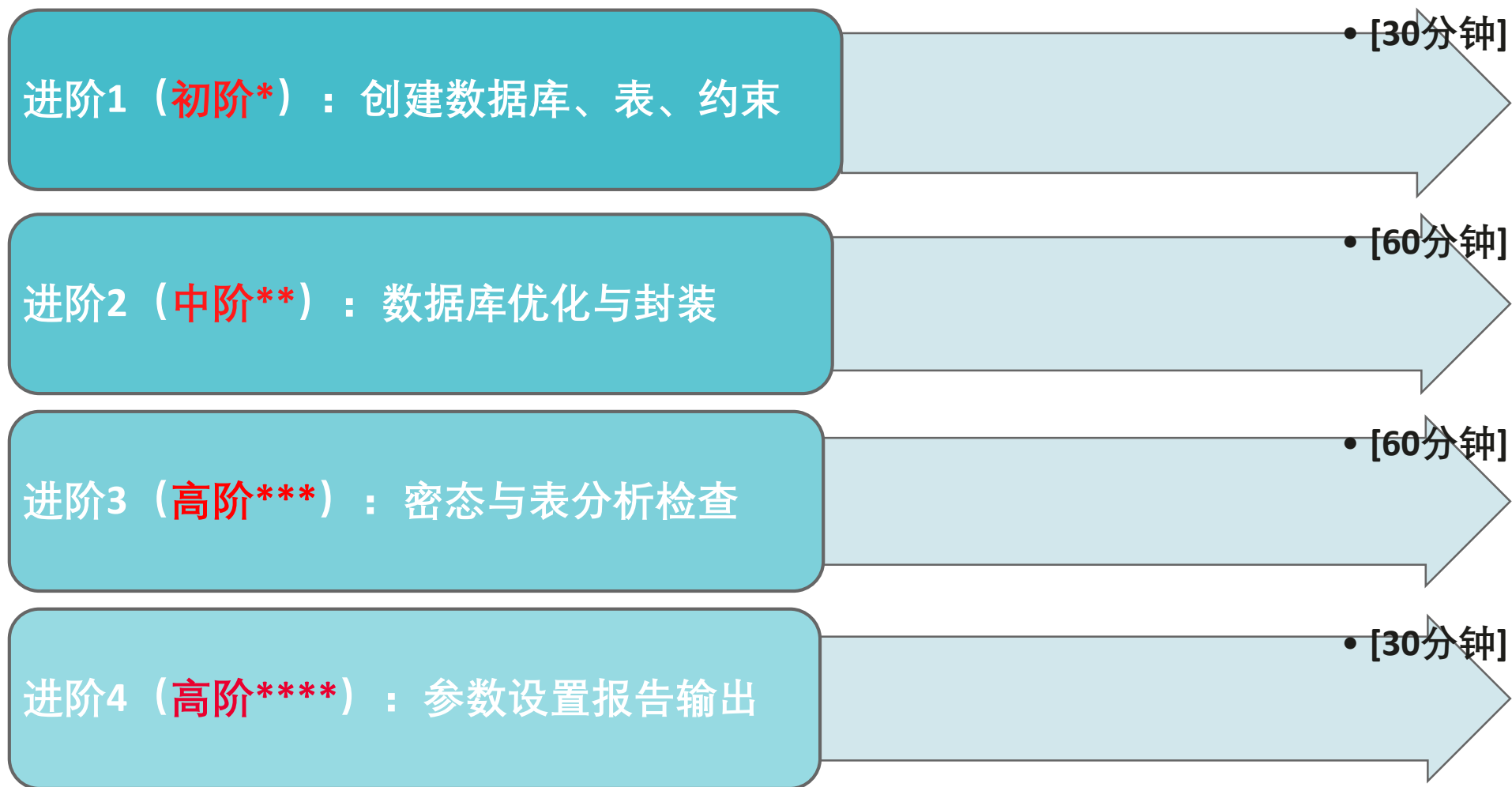
11:27 

 0

3 案例资源



4 案例实施步骤计划



5 进阶1-创建数据库、表、约束

训练要点:

视图、触发器、存储过程及事务。

需求说明:

(1) 按照数据库设计步骤对用户需求分析设计，遵循第三范式要求完成数据库结构的规范设计。

(2) 使用SQL语句创建数据库和表，并添加各种约束。

(3) 创建相应的表并建立相应的约束。

(4) 插入示例数据并实现修改密码和办理挂失业务。



5 进阶1-创建数据库、表、约束--- (1) 实验储备

第二章课后习题

本卷共分为 2大题 8小题，总分 8 分。

一、单选题(共6题，每题1分，共6分。)

1. GaussDB (for openGauss) 的三权分立开启后，查看审计日志的管理员是 ()。

☐ A 安全管理员

☐ B 存储管理员

☐ C 审计管理员

☐ D 系统管理员

2. WAL (Write Ahead Log) 日志在 openGauss 数据库中十分重要，在系统崩溃时可能用于操作恢复，其所在的默认文件夹是 ()。

☐ A pg_log

☐ B pg_xlog

☐ C pg_audit

☐ D Global

3. 启动 openGauss 数据库服务的命令是 ()。

☐ A gs_om start

☐ B start gs_om

☐ C gs_om -t start

知识模块	实验模块			
1.数据库基础	实验1.1 表空间	实验1.2 数据库模式	实验1.3 权限管理	
	实验1.4 约束	实验1.5 数据查询	实验1.6 聚合分组	
2.数据库优化和封装	实验2.1视图优化	实验2.2索引优化	实验2.3事务实现	
	实验2.4存储过程	实验2.5触发器	实验2.6游标优化	
3.数据库设计	实验3.1 E-R图设计	实验3.2 规范性设计	实验3.3 正则化设计	
	实验3.4 物理设计	实验3.5 反范式化	实验3.6 索引设计	
4.数据库开发	实验4.1 JDBC	实验4.2 Psycopy	实验4.3 ODBC	
5.数据库迁移	实验5.1 全量迁移	实验5.2 增量迁移	实验5.3 反向迁移	
6.数据库性能调优	实验6.1 索引调优	实验6.2 参数调优	实验6.3 子查询调优	实验6.4 SQL改写
7.数据库高阶特性	实验7.1 全密态数据库	实验7.2 MOT	实验7.3 DB4AI	实验7.4 社区贡献

5 进阶1-创建数据库、表、约束---（2）难点分析

（1）数据库概念结构设计和模型图的绘制。

（2）表约束的实现。

数据库原理与应用——基于openGauss

第2章 openGauss概述

第2章 openGauss概述

本章简介:

本章能力目标:

2.1 初识openGauss

2.1.1 openGauss发展史

2.1.2 openGauss核心技术

2.1.3 openGauss特性和使用场景

2.2 openGauss数据库安装与配置

2.2.1 数据库安装

2.2.2 启动与关闭

2.2.3 openGauss工具

2.2.4 Data Studio

2.3 openGauss数据库管理

2.3.1 创建和管理表空间

2.3.2 创建和管理数据库

2.3.3 数据库备份与恢复

2.4 openGauss数据库安全管理

2.4.1 用户管理

2.4.2 角色管理

2.4.3 权限管理

2.4.4 安全策略

2.5 上机练习: 学生成绩管理系统项目进阶1

1.4 关系数据库主流应用场景

1.4.1 联机事务处理

1.4.2 联机分析处理

1.4.3 数据库性能衡量指标

1.5 课后习题

第2章 openGauss概述

2.1 初识openGauss

2.1.1 openGauss发展史

2.1.2 openGauss核心技术

2.1.3 openGauss特性和使用场景

2.2 openGauss数据库安装与配置

2.2.1 数据库安装

2.2.2 启动与关闭

2.2.3 openGauss工具

2.2.4 Data Studio

2.3 openGauss数据库管理

2.3.1 创建和管理表空间

2.3.2 创建和管理数据库

2.3.3 数据库备份与恢复

2.4 openGauss数据库安全管理

2.4.1 用户管理

2.4.2 角色管理

2.4.3 权限管理

2.4.4 安全策略

2.5 上机练习: 学生成绩管理系统项目进阶1

本章简介:

本章能力目标:

2.1 初识openGauss

2.1.1 openGauss发展史

2.1.2 openGauss核心技术

2.1.3 openGauss特性和使用场景

2.2 openGauss数据库安装与配置

2.2.1 数据库安装

2.2.2 启动与关闭

2.2.3 openGauss工具

2.2.4 Data Studio

2.3 openGauss数据库管理

2.3.1 创建和管理表空间

2.3.2 创建和管理数据库

2.3.3 数据库备份与恢复

2.4 openGauss数据库安全管理

2.4.1 用户管理

2.4.2 角色管理

2.4.3 权限管理

2.4.4 安全策略

2.5 上机练习: 学生成绩管理系统项目进阶1



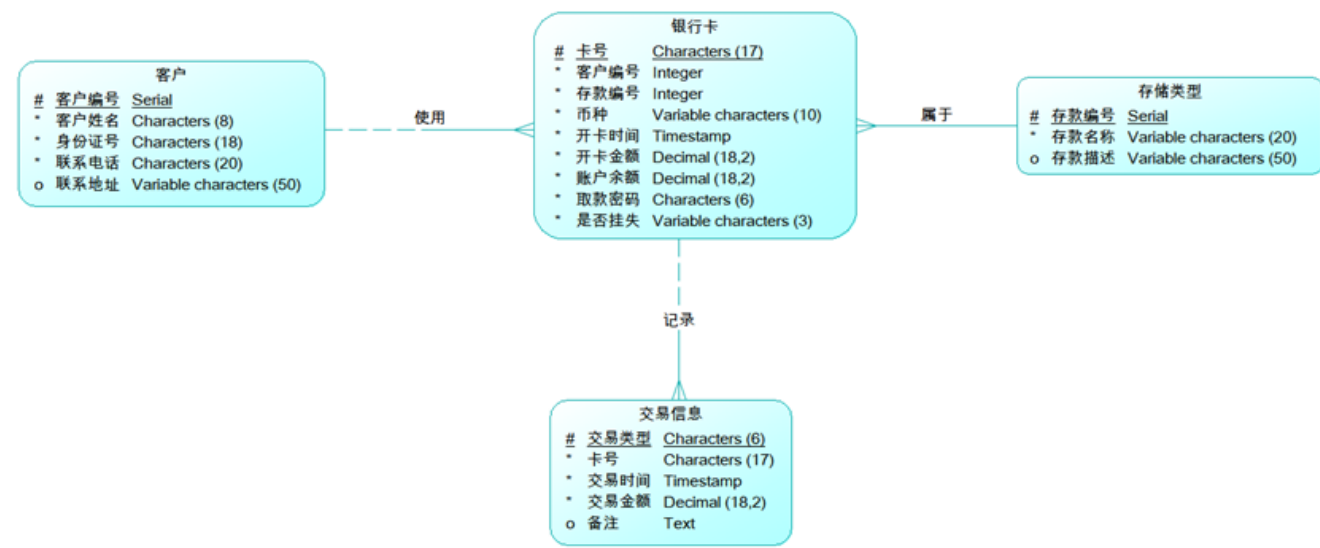
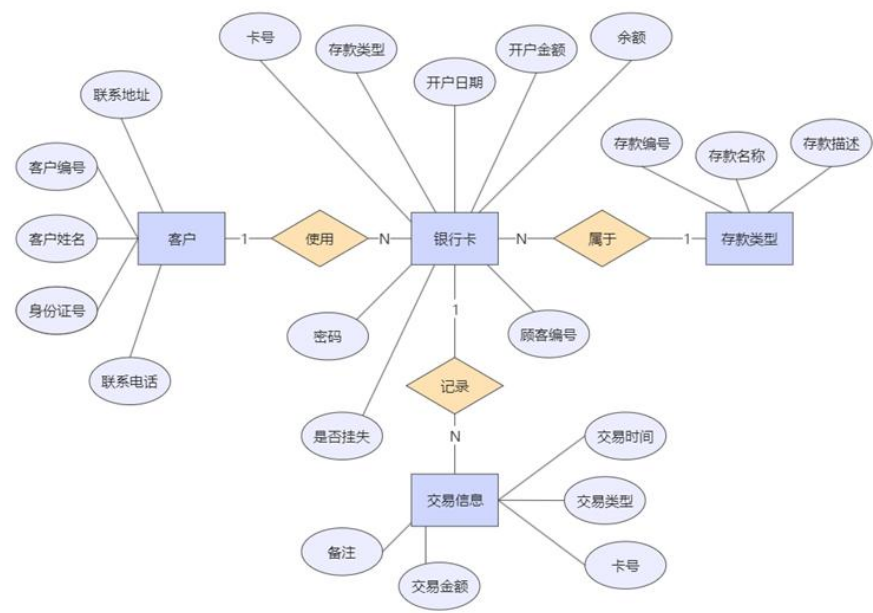
学生成绩管理系统任务1: 表空间创建、数据库与模式创建、用户与权限管理

```
CREATE DATABASE openGauss;
CREATE SCHEMA public;
CREATE TABLE public.student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    score INT NOT NULL,
    grade INT NOT NULL,
    create_time TIMESTAMP NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (id)
);
```

第三章上机练习

```
CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    score INT NOT NULL,
    grade INT NOT NULL,
    create_time TIMESTAMP NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (id)
);
```

5 进阶1-创建数据库、表、约束--- (3) E-R



5 进阶1-创建数据库、表、约束---（4）建库、建表

通用

SSL

高级

数据库类型

openGauss

名称*

bank

主机*

端口号*

最大值 65535

数据库*

bankdb

用户名*

bank_admin

密码*

●●●●●●●●

保存密码

仅当前会话

启用SSL

☐

创建四张表：分别为用户信息表、银行卡信息表、交易信息表和存款类型表

	savingid	savingname	descrip
1	1	活期	按存款日结算利息
2	2	定期一年	存款期是1年
3	3	定期二年	存款期是2年
4	4	定期三年	存款期是3年
5	5	定活两便	[NULL]
6	6	零存整取一年	存款期是1年
7	7	零存整取二年	存款期是2年
8	8	零存整取三年	存款期是3年
9	9	存本取息五年	按月支取利息

	customerid	customername	pid	telephone	address
1	1	John	11000000000000000001	13900000001	江苏省南京市
2	2	Jane	11000000000000000002	13900000002	[NULL]
3	3	Michael	11000000000000000003	13900000003	[NULL]
4	4	Emily	11000000000000000004	13900000004	[NULL]
5	5	David	11000000000000000005	13900000005	[NULL]
6	6	Sarah	11000000000000000006	13900000006	[NULL]
7	7	Robert	11000000000000000007	13900000007	[NULL]
8	8	Jessica	11000000000000000008	13900000008	[NULL]
9	9	William	11000000000000000009	13900000009	[NULL]
10	10	Olivia	11000000000000000010	13900000010	[NULL]

```
CREATE SCHEMA bank_schema;  
SET SEARCH_PATH TO bank_schema;
```

	cardid	curid	savingid	opendate	openmoney	balance	pass	isexpired	customerid
1	1010357600000001	RMB	1	2024-08-26 15:34:09	1000.00	1000.00	888888	否	1
2	1010357600000002	RMB	2	2024-08-26 15:40:08	1.00	1.00	888888	否	2
3	1010357600000003	RMB	2	2024-08-26 15:40:15	1.00	1.00	888888	否	3
4	1010357600000004	RMB	2	2024-08-26 15:40:21	1.00	1.00	888888	否	4
5	1010357600000005	RMB	3	2024-08-26 15:40:27	500.00	500.00	888888	否	5
6	1010357600000006	RMB	3	2024-08-26 15:40:33	300.00	300.00	888888	否	6
7	1010357600000007	RMB	3	2024-08-26 15:40:39	200.00	200.00	888888	否	7
8	1010357600000008	RMB	3	2024-08-26 15:40:45	150.00	150.00	888888	否	8
9	1010357600000009	RMB	3	2024-08-26 15:41:53	100.00	100.00	888888	否	9
10	1010357600000010	RMB	3	2024-08-26 15:41:58	80.00	80.00	888888	否	10

	卡号	客户姓名	货币种类	存款类型	开户日期	余额	密码	是否挂失
1	1010357600000001	John	RMB	活期	2024-08-26 15:32:32	1000.00	888888	正常
2	1010357600000002	Jane	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
3	1010357600000003	Michael	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
4	1010357600000005	David	RMB	定期二年	2024-08-26 15:32:32	500.00	888888	正常
5	1010357600000006	Sarah	RMB	定期二年	2024-08-26 15:32:32	300.00	888888	正常
6	1010357600000008	Jessica	RMB	定期二年	2024-08-26 15:32:32	150.00	888888	正常
7	1010357600000009	William	RMB	定期二年	2024-08-26 15:32:32	100.00	888888	正常
8	1010357600000010	Olivia	RMB	定期二年	2024-08-26 15:32:32	80.00	123456	正常
9	1010357600000004	Emily	RMB	定期一年	2024-08-26 15:32:32	1.00	123123	正常
10	1010357600000007	Robert	RMB	定期二年	2024-08-26 15:32:32	200.00	888888	挂失



5 进阶1-创建数据库、表、约束--- (5) 约束

用户信息表（userInfo）的约束

```
ALTER TABLE userInfo
  ADD CONSTRAINT chk_PID CHECK (CHAR_LENGTH(CAST(PID AS TEXT)) = 18),
  ADD CONSTRAINT chk_telephone CHECK
(CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone AS TEXT), '[^0-9]', '', 'g')) = 11);
```

银行卡信息表(cardInfo)的约束

```
ALTER TABLE cardInfo
  ADD CONSTRAINT ck_cardid CHECK (
  SUBSTRING(cardID, 1, 8) = '10103576'
  AND SUBSTRING(cardID, 9) ~ '^[0-9]{8}$'
),
  ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
  ADD CONSTRAINT CK_balance CHECK (balance >= 1),
  ADD CONSTRAINT CK_pass CHECK (LENGTH(pass) = 6 AND
pass ~ '^[0-9]+$'),
  ADD CONSTRAINT FK_customerID FOREIGN KEY(customerID)
REFERENCES userInfo(customerID),
  ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID)
REFERENCES deposit(savingID);
```

用交易信息表(tradeInfo)的约束

```
ALTER TABLE tradeInfo
  ADD CONSTRAINT CK_tradeType CHECK(tradeType IN
('存入', '支取')),
  ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney
> 0);
```

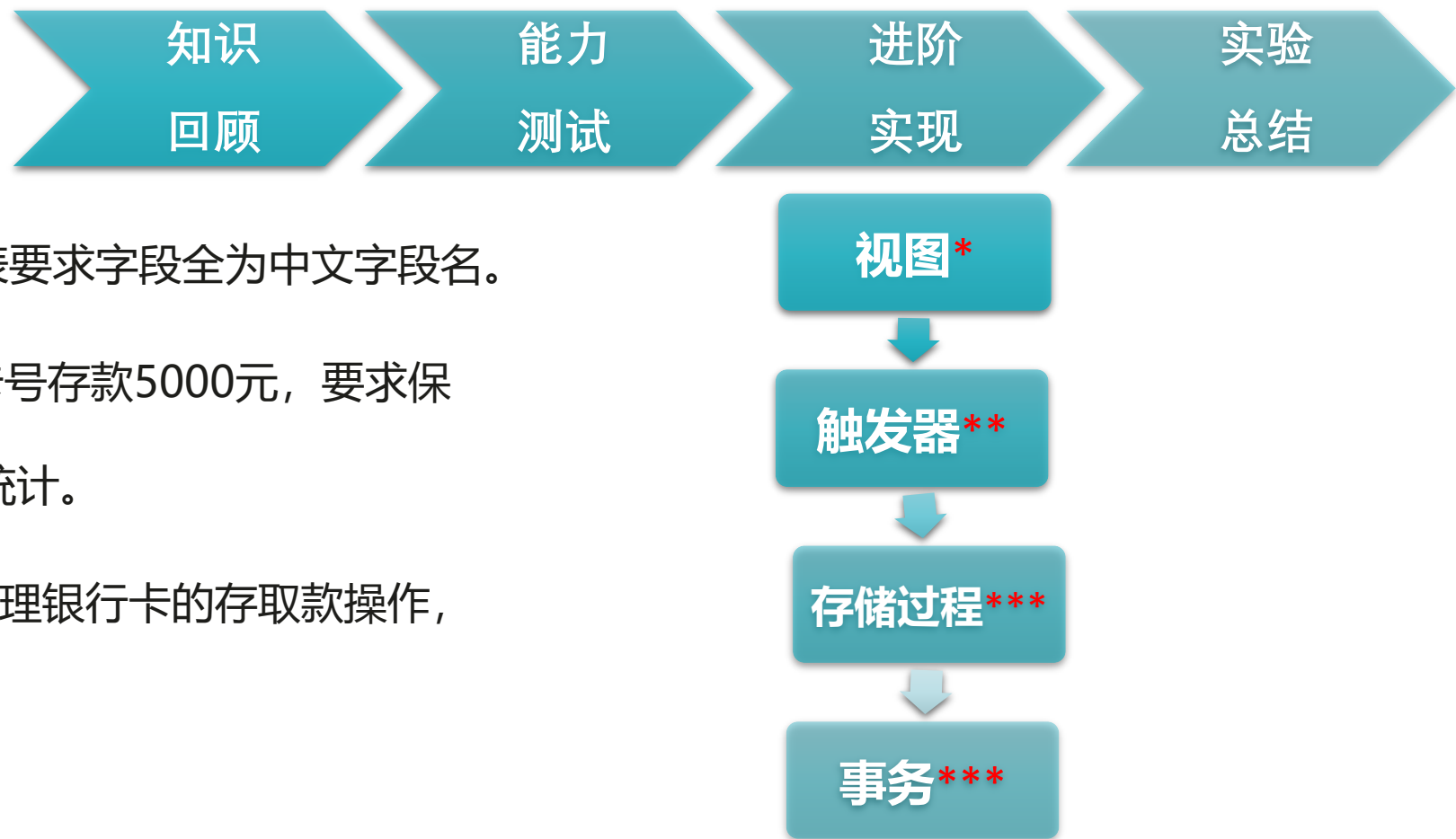
5 进阶2-数据库的业务封装

训练要点:

视图、触发器、存储过程及事务。

需求说明:

- (1) 为了向客户显示信息友好,查询各表要求字段全为中文字段名。
- (2) John的卡号取款900元, Jane的卡号存款5000元, 要求保存交易记录, 以便客户查询和银行业务统计。
- (3) 提供一个集中的、可靠的界面来管理银行卡的存取款操作, 如果存取款不成功会有错误提醒。
- (4) John向Jane转账500元。



5 进阶2-数据库的业务封装--- (1) 实验储备

本卷共分为 2 大题 7 小题，总分 7 分。

一、单选题(共5题，每题1分，共5分。)

1. 有关存储过程的参数默认值，下面说法正确的是 ()。

☐ A 输入参数必须有默认值

☐ B 带默认值的输入参数，方便用户调用

☐ C 带默认值的输入参数，用户不能再传入参数，只能采用默认值

☐ D 输出参数也可以带默认值

2. 下面有关存储过程的说法， () 是错误的。

☐ A 它可作为作为一个独立的数据库对象并作为一个单元供用户在应用程序中调用

☐ B 存储过程可以传入和返回 (输出) 参数值

☐ C 存储过程必须带参数，要么是输入参数，要么是输出参数

☐ D 存储过程提高了执行效率

3. 在 openGauss 触发器中，触发事件不包括 ()。

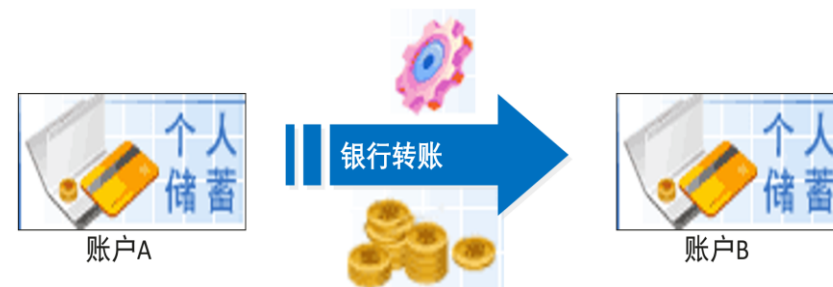
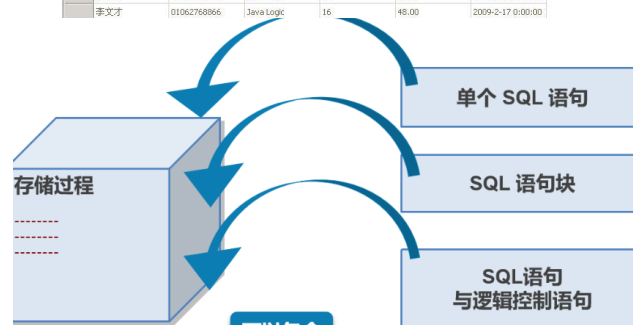
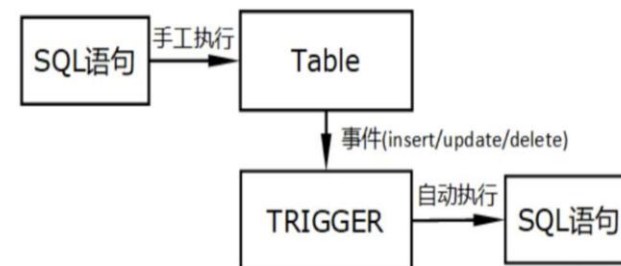
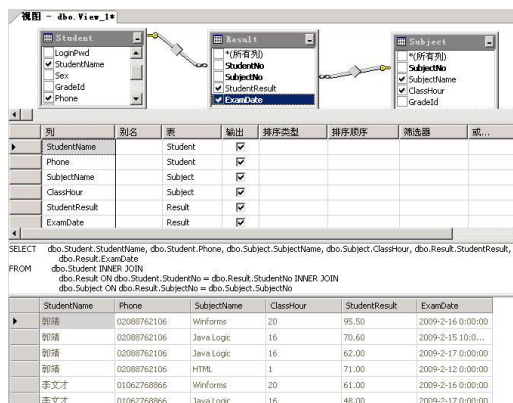
☐ A INSERT

☐ B TRUNCATE

知识模块	实验模块			
1.数据库基础	实验1.1 表空间	实验1.2 数据库模式	实验1.3 权限管理	
	实验1.4 约束	实验1.5 数据查询	实验1.6 聚合分组	
2.数据库优化和封装	实验2.1视图优化	实验2.2索引优化	实验2.3事务实现	
	实验2.4存储过程	实验2.5触发器	实验2.6游标优化	
3.数据库设计	实验3.1 E-R图设计	实验3.2 规范性设计	实验3.3 正则化设计	
	实验3.4 物理设计	实验3.5 反范式化	实验3.6 索引设计	
4.数据库开发	实验4.1 JDBC	实验4.2 Psycopy	实验4.3 ODBC	
5.数据库迁移	实验5.1 全量迁移	实验5.2 增量迁移	实验5.3 反向迁移	
6.数据库性能调优	实验6.1 索引调优	实验6.2 参数调优	实验6.3 子查询调优	实验6.4 SQL改写
7.数据库高阶特性	实验7.1 全密态数据库	实验7.2 MOT	实验7.3 DB4AI	实验7.4 社区贡献

5 进阶2-数据库的业务封装--- (2) 难点分析

- **多表关联（视图）**
- **交易更新（触发器）**
- **存取款业务（存储过程）**
- **转账异常（事务）**



⌕ ↻

testbookhelp.com.cn/preview/web/#/reader?id=P3Y3C4R5M2

☆ 100

数据库原理与应用——基于openGauss

🔍 📄 🔄 ⌂

目录

4.4.4 连接查询练习

4.4.5 聚合和分组练习

4.4.6 子查询练习

4.5 课后习题

第5章 视图、索引和事务

5.1 视图

5.1.1 视图概念

5.1.2 创建和使用视图

5.1.3 视图的作用

5.2 索引

5.2.1 索引概念

5.2.2 索引分类

5.2.3 创建和删除索引

5.3 事务

5.3.1 事务概念

本章简介:

视图可以在同一张原始数据表的基础上,为不同用户选择不同列,满足个性化需求,也可以把复杂的SQL代码进行封装,简化查询复杂度。索引能够按照指定顺序排列,从而大大提高查询速度。事务是一个工作单元,可以包含多个步骤来完成所需任务,一个事务作为一个整体,要么成功,要么失败,从而保证数据库中的数据处于同一状态。

本章能力目标:

(1) 掌握创建并使用视图的方法;

(2) 掌握创建并使用索引的方法;

(3) 使用事务保证操纵数据的完整性。

5.1 视图

5.1.1 视图概念

如表5-1所示,里面存储了该企业所有员工的详细信息,不同职位的人员查询的数据范围是不

同的。根据企业的人力资源管理要求,企业老板关注企业员工的全部信息,他可以浏览到全

体员工的全部记录;人力资源主管主要查询企业员工目前岗位、薪金和绩效;出纳人员只能查询

每个员工的薪金,不能也无权查看员工其他信息;而员工本人只能查看本人记录,不得查看其他

员工信息,那么如何实现需求呢?

视图练习例5.1

存储过程练习示例 6.9: 使用存储过程查询指定学期开设的课程数和总课时。

```

--CREATE OR REPLACE PROCEDURE get_term_course_info
--
-- 1 IN term_name VARCHAR(10),
-- 2 OUT course_count INTEGER,
-- 3 OUT total_hours NUMERIC
-- 4
-- 5 AS
-- 6 BEGIN
-- 7     -- 检查是否输入了学期名称
-- 8     IF term_name IS NULL THEN
-- 9         RAISE NOTICE '学期名称不能为空!';
-- 10        course_count := 0;
-- 11        total_hours := 0;
-- 12
-- 13        RETURN;
-- 14
-- 15 END;
  
```

0:00 / 3:22

5 进阶2-数据库的业务封装--- (3) 实验环境

知识模块

2.数据库优化和封装

实验模块

实验2.1视图优化

实验2.2触发器

实验2.3存储过程

实验2.4事务实现

实验2.5索引优化

实验2.6游标优化



训练要点：
使用可视化方式创建视图获取多表中数据。

需求说明：
需要从学生表（student）和年级表（grade）两个表中获取学生的相关信息。

关键步骤如下：

步骤一：在数据库中选择视图并右键点击创建，如图5.5所示。

步骤二：在展开的界面中录入如图5.5所示的数据。

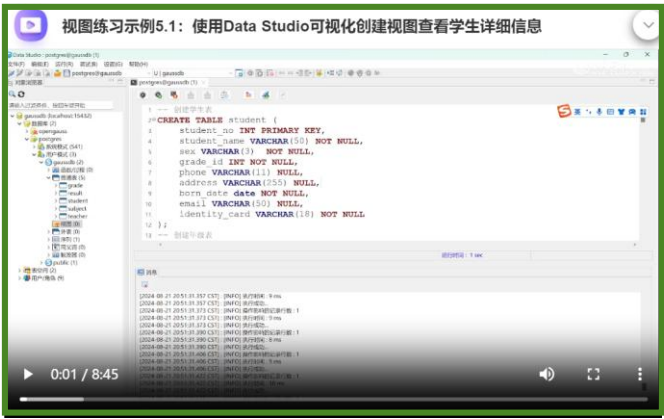
步骤三：在图5.5的下方点击“下一步”按钮，可以看到如图5.5所示的SQL语句。

步骤四：在图5.5的SQL语句进行修改（去掉了两个id列），代码如下：

```
CREATE OR REPLACE VIEW gaussdb.stuvview
AS
SELECT
    gaussdb.student.student_no AS 学号,
    gaussdb.student.student_name AS 姓名,
    gaussdb.student.sex AS 性别,
    gaussdb.student.phone AS 电话,
    gaussdb.grade.grade_name AS 年级
FROM
    gaussdb.student,
    gaussdb.grade
WHERE
    gaussdb.student.grade_id = gaussdb.grade.grade_id;
```

	学号	姓名	性别	电话	年级
1	10000	Alice	女	13900000001	大一年级
2	10001	Bob	男	13900000002	大一年级
3	10002	Charlie	女	13900000003	大一年级
4	10011	David	男	13900000004	大一年级
5	10012	Emma	女	13900000005	大一年级
6	20011	Frank	男	13900000006	大二年级
7	20012	Grace	女	13900000007	大二年级
8	30015	Hannah	女	13900000008	大三年级
9	30021	Isabella	女	13900000009	大三年级
10	40023	Jacob	男	13900000010	大四年级

第一步（实验进阶）→ 第二步（实验需求）→ 第三步（关键代码）→ 第四步（运行界面）



5 进阶2-数据库的业务封装--- (4) 视图

```
CREATE VIEW vw_cardInfo
AS
SELECT
    c.cardID AS "卡号",
    u.customerName AS "客户姓名",
    c.curID AS "货币种类",
    d.savingName AS "存款类型",
    c.openDate AS "开户日期",
    c.balance AS "余额",
    c.pass AS "密码",
    CASE WHEN c.IsReportLoss = '否' THEN '正常'
ELSE '挂失' END AS "是否挂失"
FROM cardInfo c
JOIN deposit d ON c.savingID = d.savingID
JOIN userInfo u ON c.customerID = u.customerID;
```

	卡号	客户姓名	货币种类	存款类型	开户日期	余额	密码	是否挂失
1	1010357600000001	John	RMB	活期	2024-08-26 15:32:32	1000.00	888888	正常
2	1010357600000002	Jane	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
3	1010357600000003	Michael	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
4	1010357600000005	David	RMB	定期二年	2024-08-26 15:32:32	500.00	888888	正常
5	1010357600000006	Sarah	RMB	定期二年	2024-08-26 15:32:32	300.00	888888	正常
6	1010357600000008	Jessica	RMB	定期二年	2024-08-26 15:32:32	150.00	888888	正常
7	1010357600000009	William	RMB	定期二年	2024-08-26 15:32:32	100.00	888888	正常
8	1010357600000010	Olivia	RMB	定期二年	2024-08-26 15:32:32	80.00	123456	正常
9	1010357600000004	Emily	RMB	定期一年	2024-08-26 15:32:32	1.00	123123	正常
10	1010357600000007	Robert	RMB	定期二年	2024-08-26 15:32:32	200.00	888888	挂失



5 进阶2-数据库的业务封装--- (5) 使用触发器更新余额

```
CREATE OR REPLACE FUNCTION update_balance()
```

```
RETURNS TRIGGER AS $$
```

```
DECLARE
```

```
    v_current_balance NUMERIC(18,2);
```

```
BEGIN
```

```
    IF NEW.tradeType = '存入' THEN
```

```
        UPDATE cardInfo
```

```
        SET balance = balance + NEW.tradeMoney
```

```
        WHERE cardID = NEW.cardID;
```

```
    ELSIF NEW.tradeType = '支取' THEN
```

```
        SELECT balance INTO v_current_balance
```

```
        FROM cardInfo
```

```
        WHERE cardID = NEW.cardID;
```

```
*****
```

```
CREATE TRIGGER trg_update_balance
```

```
BEFORE INSERT ON tradeInfo
```

```
FOR EACH ROW
```

```
EXECUTE PROCEDURE update_balance();
```

```
INSERT INTO tradeInfo(tradeType, cardID,  
tradeMoney)
```

```
VALUES('存入', '1010357600000005', 2000);
```

```
INSERT INTO tradeInfo(tradeType, cardID,  
tradeMoney)
```

```
VALUES('支取', '1010357600000006', 200);
```

9	1010357600000005	RMB	3	2024-08-08 15:40:27	500.00	2500.00	888888	否	5
10	1010357600000006	RMB	3	2024-08-08 15:40:33	300.00	100.00	888888	否	6

19 Huawei Confidential

5 进阶2-数据库的业务封装--- (6) 使用存储过程实现业务处理

完成存款或取款业务

CREATE OR REPLACE PROCEDURE

deposit_withdraw(

p_cardid CHAR(19),

p_money NUMERIC(18,2),

p_trade_type CHAR(4)

)

AS

v_balance NUMERIC(18,2);

BEGIN

-- 检查卡号是否存在

IF NOT EXISTS (SELECT 1 FROM cardInfo

WHERE cardID = p_cardid) THEN

RAISE_APPLICATION_ERROR(20001, '输入不符合规范');

END IF;

CALL deposit_withdraw('10103576000000001', 1000.00, '存入'); --正常执行

CALL deposit_withdraw('10103576000000002', 500.00, '支取'); --正常执行

CALL deposit_withdraw('10103576000000011', 500.00, '支取'); --错误的卡号

CALL deposit_withdraw('10103576000000002', 5000.00, '支取'); --支取的金额超出余额

CALL deposit_withdraw('10103576000000002', 500.00, '取款'); --错误的输入类型

查询结果为：执行成功

执行成功

ERROR:卡号不存在

ERROR:金额不足

ERROR:输入不符合规范

5 进阶2-数据库的业务封装--- (6) 使用存储过程实现业务处理

```
产生随机卡号
CREATE OR REPLACE PROCEDURE
usp_randCardID(OUT randCardID VARCHAR(17))
AS
BEGIN
    -- 生成整数随机数种子，并确保其在可接受的范围内
    r1 := CAST(RANDOM() * 9999999 AS BIGINT); --
    确保随机数长度不超过7位
    r2 := CAST(RANDOM() * 9999999 AS BIGINT);
    -- 转换为字符串
    tempStr1 := LPAD(CAST(r1 AS VARCHAR), 10, '0');
    -- 保证字符串长度为10，不足部分左侧补零
    *****
```

```
DO $$
DECLARE
    mycardID VARCHAR(17);
BEGIN
    mycardID := usp_randCardID(); -- 直接调用函数并赋值
    RAISE NOTICE '产生的随机卡号为: %', mycardID;
END $$;
```

查询结果为：产生的随机卡号为：
10103576xxxxxxxxxx。

5 进阶2-数据库的业务封装--- (6) 使用存储过程实现业务处理

完成开户业务

```
CREATE OR REPLACE PROCEDURE
usp_openAccount(
    IN p_customerName VARCHAR(8),
    IN p_PID CHAR(18),
    IN p_telephone CHAR(11),
    IN p_openMoney NUMERIC(18,2),
    IN p_savingName CHAR(8),
    IN p_address VARCHAR(50) DEFAULT "
)
AS
DECLARE
    v_cardID VARCHAR(19);
    v_customerID INTEGER;
    v_savingID INTEGER;
BEGIN *****
```

```
CALL usp_openAccount('Emma',
'1100000000000000011', '13900000011', 1000.00, '活期', '辽
宁省大连市');
```

查询结果为：尊敬的客户，开户成功！系统为您产生的随机卡号为：10103576XXXXXXXXX 开户日期202X-XX-XX 开户金额：1000.00

5 进阶2-数据库的业务封装--- (6) 使用存储过程实现业务处理

分页显示查询交易数据

CALL usp_pagingDisplay();

```
CREATE OR REPLACE PROCEDURE
usp_pagingDisplay(
    IN p_page INT = 1,
    IN p_records INT = 10
)
AS
DECLARE
    v_offset INT;
    v_tradeDate TEXT;
    v_tradeType TEXT;
    v_cardID TEXT;
    v_tradeMoney NUMERIC;
    *****
```

```
[NOTICE] 交易日期: 2024-08-08 15:52:59.189046, 交易类型: 支取, 卡号: 1010357600000001, 交易金额: 900.00
[NOTICE] 交易日期: 2024-08-08 15:53:02.45374, 交易类型: 存入, 卡号: 1010357600000002, 交易金额: 5000.00
[NOTICE] 交易日期: 2024-08-08 16:03:15.151009, 交易类型: 存入, 卡号: 1010357600000003, 交易金额: 300.00
[NOTICE] 交易日期: 2024-08-08 16:03:53.412365, 交易类型: 存入, 卡号: 1010357600000004, 交易金额: 1000.00
[NOTICE] 交易日期: 2024-08-08 16:05:04.042534, 交易类型: 支取, 卡号: 1010357600000002, 交易金额: 1900.00
[NOTICE] 交易日期: 2024-08-08 16:38:41.058165, 交易类型: 存入, 卡号: 1010357600000005, 交易金额: 2000.00
[NOTICE] 交易日期: 2024-08-08 16:39:33.248524, 交易类型: 支取, 卡号: 1010357600000006, 交易金额: 200.00
[NOTICE] 交易日期: 2024-08-08 16:51:38.158797, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
[NOTICE] 交易日期: 2024-08-08 16:54:02.374043, 交易类型: 支取, 卡号: 1010357600000002, 交易金额: 500.00
[NOTICE] 交易日期: 2024-08-08 16:56:57.774065, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
```

5 进阶2-数据库的业务封装--- (7) 使用事务实现客户转账

本实验通过创建事务实现John（卡号为1010357600000001）向Jane（卡号为1010357600000002）转账500元。

```
BEGIN ;  
INSERT INTO tradeInfo (tradeType, cardID,  
tradeMoney, remark)  
VALUES ('支取', '1010357600000001', 500, '转账扣款'  
);  
INSERT INTO tradeInfo (tradeType, cardID,  
tradeMoney, remark)  
VALUES ('存入', '1010357600000002', 500, '转账收款  
来自John');  
END;
```

12	2024-08-26 20:37:09	支取	1010357600000001	500.00	转账扣款
13	2024-08-26 20:37:13	存入	1010357600000002	500.00	转账收款来自John

5 进阶3-密态与表分析检查

本次进阶实验主要完成以下内容。

- (1)采用全密态数据库和防篡改模式，保证数据在全生命周期中的加密状态，确保银行业务系统的完整性和可信度。
- (2)检查的网络、设备、操作系统、数据库等类别的所有检查项目，并执行所有的数据库检查项目，修正检查中发现的问题。
- (3)检查openGauss集群的主要性能指标。

知识模块	实验模块			
1.数据库基础	实验1.1 表空间	实验1.2 数据库模式	实验1.3 权限管理	
	实验1.4 约束	实验1.5 数据查询	实验1.6 聚合分组	
2.数据库优化和封装	实验2.1视图优化	实验2.2索引优化	实验2.3事务实现	
	实验2.4存储过程	实验2.5触发器	实验2.6游标优化	
3.数据库设计	实验3.1 E-R图设计	实验3.2 规范性设计	实验3.3 正则化设计	
	实验3.4 物理设计	实验3.5 反范式化	实验3.6 索引设计	
4.数据库开发	实验4.1 JDBC	实验4.2 Psycopy	实验4.3 ODBC	
5.数据库迁移	实验5.1 全量迁移	实验5.2 增量迁移	实验5.3 反向迁移	
6.数据库性能调优	实验6.1 索引调优	实验6.2 参数调优	实验6.3 子查询调优	实验6.4 SQL改写

5 进阶3-密态与表分析检查--- (1) 使用全密态数据库实现数据加密

实现思路和关键代码:

步骤1: 在omm用户下创建\$GAUSSHOMe/etc/localkms/路径。

mkdir -p \$GAUSSHOMe/etc/localkms/

步骤2: omm用户登录bankdb数据库, 并开启密态开关。

gsql -d bankdb -p 26000 -r -C

步骤3: 在bankdb下创建防篡改模式bank_security。

CREATE SCHEMA bank_security WITH BLOCKCHAIN;

SET SEARCH_PATH TO bank_security;

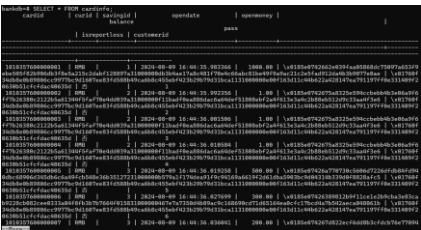
步骤4: 创建客户端主密钥CMK和列加密密钥CEK。

CREATE CLIENT MASTER KEY ImgCMK01 WITH (KEY_STORE = localkms, KEY_PATH = "key_path_value1", ALGORITHM = RSA_2048);

步骤5: 在bank_security模式下创建加密表。

CREATE TABLE userInfo

(
customerID SERIAL PRIMARY KEY,
customerName CHAR(8) NOT NULL,
PID CHAR(18) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK01, ENCRYPTION_TYPE =



5 进阶3-密态与表分析检查--- (2) 使用gs_check实现数据表分析

本实验获取gs_check所能够检查的网络、设备、操作系统、数据库等类别的所有检查项目，并执行所有的数据库检查项目，修正检查中发现的问题，比如数据表没有进行analyze。

步骤1：获取gs_check所在的路径。

步骤2：把下面命令中的/opt/huawei/wisquery/script替换为上面命令返回的gs_check的目录名，获取gs_check所支持的所有检测项目。

步骤3：创建目录mkdir /home/omm/lab。

步骤4：将所有数据库检测项目保存到文件 /home/omm/lab/check_db_list。

步骤5：执行命令，实现对所有数据库检测项目的检测。

步骤6：对CheckUnAnalyzeTable所返回的bankdb数据库的表进行analyze操作，以omm用户登录到bankdb数据库上。

步骤7：在bank_security模式下对四张表进行执行analyze操作。

步骤8:重新对项目CheckUnAnalyzeTable执行gs_check。可以看到，bank_security模式下对表都已经做了analyze，没有出现在返回结果中了。

5 进阶3-密态与表分析检查--- (3) 使用gs_checkperf实现性能检查

本实验使用gs_checkperf工具的两种模式，检查openGauss集群的主要性能指标。

步骤1：以omm用户运行gs_checkperf命令，显示当前集群的简单信息。

步骤2：以omm用户运行gs_checkperf命令，显示当前集群的详细信息。

```
[omm@ecs-6998-3849-7352 lab]$ gs_checkperf
Cluster statistics information:
  Host CPU busy time ratio           :      2.27      %
  MPPDB CPU time % in busy time     :      17.54     %
  Shared Buffer Hit ratio             :      99.65     %
  In-memory sort ratio               :           0
  Physical Reads                     :      67455
  Physical Writes                    :       827
  DB size                           :      1005      MB
  Total Physical writes              :       827
  Active SQL count                   :           4
  Session count                      :           8
```

5 进阶3-密态与表分析检查--- (4) 使用gs_collector收集日志

本实验使用gs_collector收集OS信息、日志信息以及配置文件等信息，并查看每个日志的内容。

步骤1: 执行gs_collector命令，请修改时间戳为当前合适的值。

步骤2: 解压缩收集到的压缩文件。

步骤3: 进入解压缩目录并再次解压。

步骤4: 查看收集到的操作系统的状态信息。

```
[omm@ecs-6998-3849-7352 lab]$ gs_collector --begin-time='20240809 07:00' --end-time='20230809 23:00'
[GAUSS-50004] : The parameter '--end-time or --begin-time' value is incorrect.The value of '--end-time' must be greater
than the value of '--begin-time'.
[omm@ecs-6998-3849-7352 lab]$ gs_collector --begin-time='20240809 07:00' --end-time='20240809 23:00'
Successfully parsed the configuration file.
create Dir.
Successfully create dir.
do system check interval 0 : count 1
Collecting OS information.
The cmd is source /home/omm/.bashrc; python3 '/opt/huawei/wisquery/script/local/LocalCollect.py' -t system_check -U omm
-l /var/log/gaussdb/omm/om/gs_local.log -C '{#TypeName#: #System#, #Content#: #ps,ioStat,netFlow,spaceUsage,cpuInfo,mem
Info,disk,#, #Interval#: #0#, #Count#: #1#}'
Failed to collect OS information.
do database check interval 0 : count 1
Collecting catalog statistics.
Successfully collected catalog statistics.
do log check interval 0 : count 1
Collecting Log files.
Successfully collected Log files.
do Config check 0:1
Collecting Config files.
Successfully collected Config files.
Collecting files.
Successfully collected files.
All results are stored in /var/log/gaussdb/omm/collector_20240809_224357.tar.gz.
```

5 进阶4-参数设置报告输出

本次进阶实验主要完成以下内容。

- (1) 能够设置数据库的参数，调整数据库的性能。
- (2) 掌握WDR报告的生成流程，学会解读WDR报告。

知识模块	实验模块			
1.数据库基础	实验1.1 表空间	实验1.2 数据库模式	实验1.3 权限管理	
	实验1.4 约束	实验1.5 数据查询	实验1.6 聚合分组	
2.数据库优化和封装	实验2.1视图优化	实验2.2索引优化	实验2.3事务实现	
	实验2.4存储过程	实验2.5触发器	实验2.6游标优化	
3.数据库设计	实验3.1 E-R图设计	实验3.2 规范性设计	实验3.3 正则化设计	
	实验3.4 物理设计	实验3.5 反范式化	实验3.6 索引设计	
4.数据库开发	实验4.1 JDBC	实验4.2 Psycopy	实验4.3 ODBC	
5.数据库迁移	实验5.1 全量迁移	实验5.2 增量迁移	实验5.3 反向迁移	
6.数据库性能调优	实验6.1 索引调优	实验6.2 参数调优	实验6.3 子查询调优	实验6.4 SQL改写
7.数据库高阶特性	实验7.1 全密态数据库	实验7.2 MOT	实验7.3 DB4AI	实验7.4 社区贡献

5 进阶4-参数设置报告输出----- (1) 调整shared buffer参数

本实验结合操作系统的内存大小，设置数据库shared buffer的大小。

步骤1：查看当前操作系统的总内存和空闲内存。

步骤2：使用gsql客户端以管理员用户身份连接bankdb数据库。

步骤3：查看当前数据库shared buffer的大小。

步骤4：调整shared buffer参数。

步骤5：停止数据库、再次启动数据库、验证数据库。

```
[omm@ecs-6998-3849-7352 ~]$ gsql -d bankdb -p 26000 -r -c 'show shared_buffers'
shared_buffers
-----
10GB
(1 row)
```

5 进阶4-参数设置报告输出-----（2）使用快照生成WDR报告

本实验基于两个快照，生成WDR报告，并进行解读。

步骤 1以omm用户登录数据库postgres。

步骤 2显示enable_wdr_snapshot的值。

步骤 3若enable_wdr_snapshot为off，使用以下命令设置为on。

步骤 4查询当前存在的快照。

步骤5： 停止数据库、再次启动数据库、验证数据库。

步骤 6若当前只有一个快照，重建出一个新快照。

步骤 7利用最近的两个快照，生成一份WDR报告。

Workload Diagnosis Report

Report Type	Report Scope	Report Node			
Summary + Detail Node		dn_6001			
Snapshot Id	Start Time	End Time			
1	2024-08-09 23:30:15	2024-08-09 23:30:16			
2	2024-08-09 23:33:16	2024-08-09 23:33:16			
Host Node Name	CPUs	Cores	Sockets	Physical Memory	openGauss Version
dn_6001	8	8		1.31 GB	(openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mtr on aarch64-unknown-linux-gnu, compiled by g++ (GCC) 7.3.0, 64-bit

Summary

-Instance Efficiency Percentages (Target 100%)

• Get instance efficiency percentages.

Metric Name	Metric Value
Buffer Hit %	99.33
Effective CPU %	95
WaitWrite NoWait %	100
Soft Parse %	11
Non-Parse CPU %	99

[Back to Summary](#)

[Back to Top](#)

-Top 10 Events by Total Wait Time

• Top 10 Events by Total Wait Time

Event	Waits	Total Wait Time(us)	Avg Wait Time(us)	Type
BufHashTableSearch	107168	187227		1 IO_EVENT
DoubleWriteFileWrite	10	39135		3914 IO_EVENT
HashAgg - build hash	24	33384		1391 STATUS
DoubleWriteLock	4	28688		7172 LWLOCK_EVENT
ControlFileSyncUpdate	6	9681		1614 IO_EVENT
Flush data	294	6624		23 STATUS
DataFileSync	150	5211		35 IO_EVENT
SLRUFlushSync	2	4199		2100 IO_EVENT
DataFileExtend	380	3189		8 IO_EVENT
WALWrite	253	2936		12 IO_EVENT

[Back to Summary](#)

[Back to Top](#)

6 实验总结



全国有6万人参与闯关



 学霸榜  数据库系统闯关排行榜			
姓名	所在院校	闯关个数	闯关积分
王		8	9800
张		8	9800
李		8	9800
王		8	9800
王		8	9800
王		8	9800
张		8	9800
沈		8	9800
虞		7	9600
郝		7	9600
王		7	9600
杨		7	9600
王		7	9600
范		7	9600
李		7	9600
王		7	9600
张		7	9600
王		8	9800
张		8	9800
李		8	9800
王		8	9800
王		8	9800
王		8	9800
张		8	9800
沈		8	9800
虞		7	9600
郝		7	9600
王		7	9600
杨		7	9600
王		7	9600
范		7	9600
李		7	9600
王		7	9600
张		7	9600

比—>学—>赶—>帮—>超

6 案例实验总结

- 能够系统学习数据库的理论和实践

- 1、能够完成1个完整的数据库项目

- 2、能够参加1项openGauss认证

- 3、能够参加1项ICT竞赛

- 4、能够成为1名优秀开发者

$$1 + 1 + 1 + 1 > 4$$

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home and
organization for a fully connected,
intelligent world.

**Copyright©2018 Huawei Technologies Co., Ltd.
All Rights Reserved.**

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

